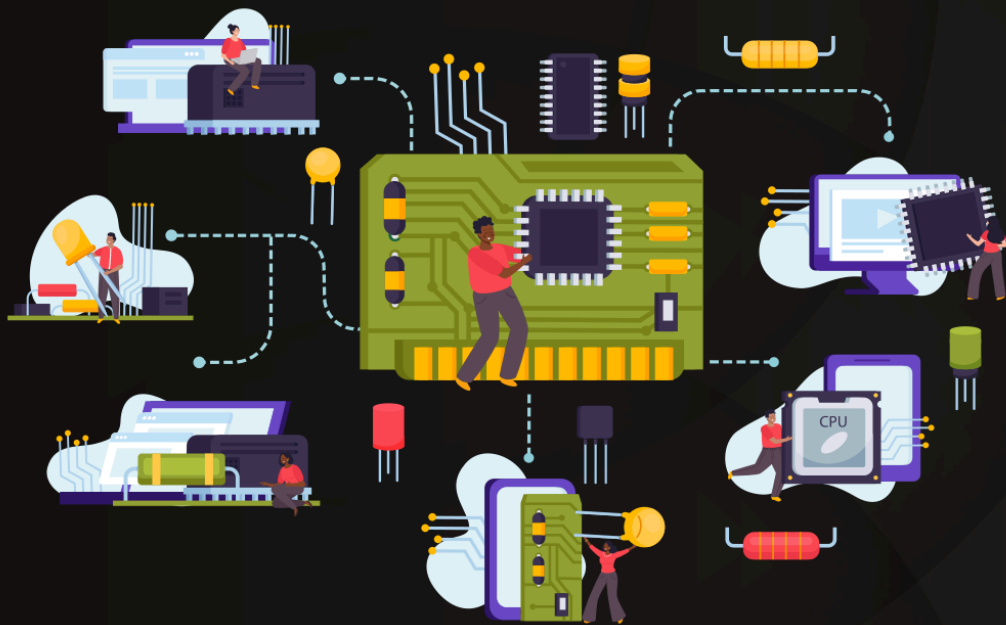


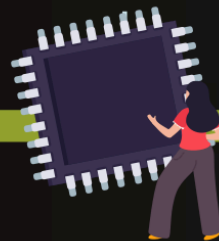
SEMINÁRIO DE SISTEMAS EMBARCADOS E IOT 2026



Zephyr RTOS: Praticidade para quem desenvolve hardware e software



Matheus Marques



Agenda

- O que é?
- Aplicação prática
- Bibliotecas e configurações
- Como implemento "x"?
- Devo começar a utilizar?



Apresentação



Formação e experiência

Graduado em **Engenharia Eletrônica** pela Universidade Federal de Itajubá (UNIFEI)

Desenvolvimento de **projetos de hardware, software embarcado** e fluxos **DevSecOps**

- Microcontroladores 8bits e Cortex Arm M
- FreeRTOS
- Linux Embarcado
- Yocto Project
- Zephyr RTOS

Desenvolvedor de software embarcado na **Padtec**, trabalhando com equipamentos de rede na camada óptica





MATHEUS MARQUES

I'm an Electronic Engineer always trying to learn more about developing embedded systems and help other people to do so



embedded firmware from scratch - part 1

Starting a new firmware development project requires multiple decisions, even after the microcontroller and hardware are already selected. One of the first steps is defining the development environment, that in my conception includes:

- code versioning system (git)
- language (C or C++)
- toolchain (compiler, linker, assembler)
- build system (shell, make, ninja, CMake)
- debugger

For many years I've let the microcontroller vendor's IDE take care of most of these decisions. Now I've decided to challenge myself and create a new firmware project from scratch, without using an IDE, in order to learn what takes to make everything work together. This has required learning about multiple tools and the workflow to combine their functions.

This is the **first article** about this endeavor, focusing on a few obstacles that arise due using vendor tools (IDE, toolchain, libraries, etc) instead of assuming full control of your project and setting it up from scratch.

[Continue reading...](#)

AUGUST 29TH, 2022

getting a Black Magic Probe

I've heard recently in a C++ conference talk that:

C programmers love debugging, they do all the code just to get to the debugging part".

Am I going to disagree with that? Yeah, probably.

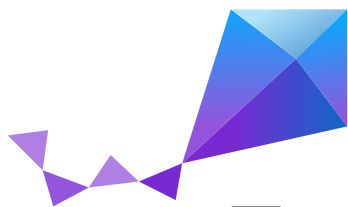
- applying TDD and intelligent development practices intend to help on that
- good tests will avoid a lot of time spent on debugging directly in the target

matheusmbar.com 



cursos.embarcados.com.br 





Zephyr[®]

O que é?



Zephyr RTOS

É em sistema operacional de tempo real (RTOS):

- pequeno
- escalável
- otimizado para dispositivos com recursos limitados
- suporte a múltiplas arquiteturas

Projeto open-source colaborativo gerenciado pelas Linux Foundation.

Recebe contribuição de diversas empresas fabricantes de componentes e produtos.



Zephyr RTOS | Members

Platinum Members

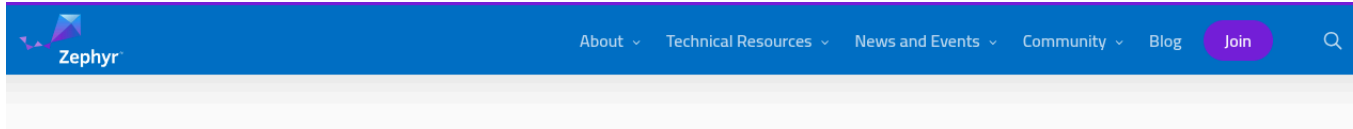


WINDRVR

Silver Members

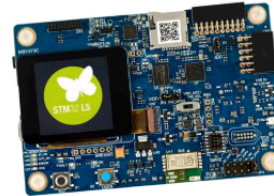
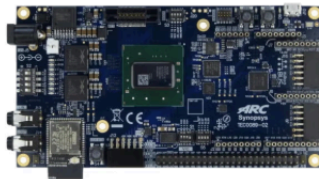


Zephyr RTOS | +1000 boards



1000+ Boards Supported

Zephyr supports more than 1000 boards. Search [our list](#) for the hardware used in your application. This diversity of supported boards gives developers and product manufacturers multiple options to solve their embedded RTOS challenges with Zephyr. If your board is not supported out of the box, adding support for a new board is simple.



Aplicação prática



Aplicação básica com Zephyr

- Arquivos de configuração da compilação e build system
- Um source code para a `main()`

```
$ tree -L2
.
├── CMakeLists.txt
├── prj.conf
├── README.md
├── sample.yaml
└── src
    └── main.c
```



Aplicação básica com Zephyr

```
// src/main.c
#include <zephyr/logging/log.h>

LOG_MODULE_REGISTER(main);

int main() {
    LOG_INF("Hello from %s", CONFIG_BOARD);
    return 0;
}
```

```
*** Booting Zephyr OS build v4.4.0 ***
[00:00:00.008,955] <inf> main: Hello from frdm_mcxa156
iot-2026:~$ _
```

Terminal iterativo na UART

- Configuração de GPIOs
- Leitura de ADC
- SPI config, transfer
- I2C scan, read, write
- I3C info, read, write
- Leitura de sensores
- Escrita em DAC
- Configuração de rede
- Ping
- Consulta e controle de threads
- Suporta múltiplos targets
- Compila para todos com um comando



Zephyr SHELL

Terminal iterativo no
microcontrolador



Zephyr SHELL - help

```
iot-2026:~$ help
Please press the <Tab> button to see all available commands.
You can also use the <Tab> button to prompt or auto-complete all commands
or its subcommands.
You can try to call commands with <-h> or <--help> parameter for more information.
```

Shell supports following meta-keys:

Ctrl + (a key from: abcdefklntuw)

Alt + (a key from: bf)

Please refer to shell documentation for more details.

Available commands:

```
adc      : ADC commands
clear    : Clear screen.
dac      : DAC shell commands
date     : Date commands
device   : Device commands
```



Zephyr SHELL - help

```
devmem      : Read/write physical memory
              Usage:
              Read memory at address with optional width:
              devmem <address> [<width>]
              Write memory at address with mandatory width and value:
              devmem <address> <width> <value>

gpio        : GPIO commands
help        : Prints the help message.
history     : Command history.
i2c         : I2C commands
i3c         : I3C commands
input       : Input commands
kernel      : Kernel commands
led         : LED commands
net         : Networking commands
retval      : Print return value of most recent command
sensor      : Sensor commands
spi         : SPI commands
```



GPIO

```
iot-2026:~$ gpio [TAB]
  conf    get    set    toggle  devices  blink  info
iot-2026:~$ gpio set [TAB]
  gpio0  gpio1  gpio2  gpio3  gpio4
```

```
# configurar como OUTPUT
iot-2026:$ gpio conf gpio3 12 o
```

```
# Set level - HIGH - LOW
iot-2026:$ gpio set gpio3 12 1
iot-2026:$ gpio set gpio3 12 0
```

```
# configurar pino do Switch como INPUT
iot-2026:$ gpio conf gpio1 7 i
```

```
# Get level
iot-2026:$ gpio get gpio1 7
1
iot-2026:$ gpio get gpio1 7
0
```

Suporte a GPIOS:

- nativos
- conectados em expensor de I/O



LED

```
iot-2026:~$ led [TAB]
  off          on          blink      get_info
  set_brightness set_color  set_channel write_channels
```

```
iot-2026:$ led on leds 1
leds: turning on LED 1
```

```
iot-2026:$ led on leds 2
leds: turning on LED 2
```

```
iot-2026:$ led off leds 1
leds: turning off LED 1
```

```
iot-2026:$ led off leds 2
leds: turning off LED 2
```

LEDs conectados em:

- GPIOs nativos
- expensor de I/O
- controlador de LED

Controle de PWM



ADC / DAC

```
iot-2026:~$ adc [TAB]
lpadc@400af000

iot-2026:~$ adc lpadc@400af000 [TAB]
acq_time      channel      gain      print
read          reference    resolution

iot-2026:$ adc lpadc@400af000 resolution 12

iot-2026:$ adc lpadc@400af000 read 0
read: 220

iot-2026:$ adc lpadc@400af000 read 1
read: 228

iot-2026:$ adc lpadc@400af000 read 2
read: 319
```

```
iot-2026:~$ dac [TAB]
setup          write_value

iot-2026:~$ dac setup [TAB]
dac@400b4000

iot-2026:~$ dac setup dac@400b4000 0 12

iot-2026:~$ dac write_value dac@400b4000 0 100
```

Canais de ADC/DAC:

- nativos
- CIs externos (I2C, SPI, etc)



I2C

```
iot-2026:$ i2c [TAB]
  scan      recover      read      read_byte  direct_read
  write      write_byte  speed
iot-2026:$ i2c s [TAB]
  scan      speed
iot-2026:$ i2c scan [TAB]
  i2c@400d5000  i2c@4009a000

iot-2026:$ i2c scan i2c@400d5000
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- 36 -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- 59 -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
2 devices found on i2c@400d5000
```



I3C

```
iot-2026:$ i3c
  info      speed      recover
  read      read_byte  write
  write_byte i3c_attach  i3c_reattach
  i3c_detach i2c_attach  i2c_detach
  i2c_scan   ccc      hdr ibi
```

```
iot-2026:$ i3c info i3c@40002000
I3C: Devices found:
name: p3t1755@48000000236152a0090
  pid: 0x0236152a0090
  static_addr: 0x48
  dynamic_addr: 0x48
  bcr: 0x03
  dcr: 0x63
  maxrd: 0x00
  maxwr: 0x00
  max_read_turnaround: 0x000000
  mrl: 0x0000
  mwl: 0x0000
  max_ibi: 0x00
  crhdly1: 0x00
  getcaps: 0x00; 0x00; 0x00; 0x00
  crcaps: 0x00; 0x00
I2C: No devices found.
```



Sensores

```
iot-2026:$ sensor [TAB]
  get      attr_set      attr_get      stream      trig

# Leitura
iot-2026:$ sensor get p3t1755@48000000236152a0090
channel type=13(ambient_temp) index=0 shift=5 num_samples=1 value=388890770000ns (26.812500)

iot-2026:$ sensor get p3t1755@48000000236152a0090
channel type=13(ambient_temp) index=0 shift=5 num_samples=1 value=391552798000ns (26.750000)
```

Target: FRDM MCX156A

Sensor de Temperatura Ambiente P3T1755 conectado ao I3C



Sensores

```
# Acelerômetro
iot-2026:~$ sensor get fxos8700@1c acc [TAB]
  accel_x    accel_y    accel_z    accel_xyz

iot-2026:~$ sensor get fxos8700@1c accel_xyz
channel type=3(accel_xyz) index=0 shift=4 num_samples=1 value=122307504600ns,
  (-1.302446, -0.105345, 9.787495)

# Magnetômetro
iot-2026:~$ sensor get fxos8700@1c mag [TAB]
  magn_x    magn_y    magn_z    magn_xyz

iot-2026:~$ sensor get fxos8700@1c magn_xyz
channel type=11(magn_xyz) index=0 shift=0 num_samples=1 value=136067275050ns,
  (0.588999, -0.131999, 0.235999)
```

Target: FRDM K82F

Acelerômetro e Magnetômetro FXOS8700 conectado ao I2C



SPI / DSPI / QSPI

```
iot-2026:$ spi conf -h
conf - Configure SPI bus
      Usage: conf <spi-device> <frequency> [<settings>]
            <settings> any sequence of letters:
            o - SPI_MODE_CPOL
            h - SPI_MODE_CPHA
            l - SPI_TRANSFER_LSB
            T - SPI_FRAME_FORMAT_TI
            Example: spi conf spi1 1000000 ol

iot-2026:$ spi cs -h
cs - Assign CS GPIO to SPI device
     Usage: cs <spi-device> <gpio-device> <pin> [<flags>]
           Example: spi cs spi1 gpio1 3 0x01

iot-2026:$ spi transceive -h
transceive - Transceive data to/from SPI device
            Usage: transceive <spi-device> <tx-byte1> [<tx-byte2> ...]
            Example: spi transceive spi1 0x00 0x01
```



Network

```
iot-2026:~$ net -h
net - Networking commands
Subcommands:
  allocs      : Print network memory allocations.
  arp         : Print information about IPv4 ARP cache.
  capture     : Configure network packet capture.
  cm          : Control conn_mgr.
  conn        : Print information about network connections.
  dhcpv4      : Manage DHCPv4 services
  dhcpv6      : Manage DHCPv6 services.
  dns         : Show how DNS is configured. Optionally do a query using a given name.
  events      : Monitor network management events.
  filter      : Print information about network packet filters.
  ftp         : FTP client commands
  gptp        : Print information about gPTP support.
  http        : Show HTTP services.
  iface       : Print information about network interfaces.
  ipv4        : Print information about IPv4 specific information and configuration.
  ipv6        : Print information about IPv6 specific information and configuration.
```



Network

```
iot-2026:~$ net -h
mem      : Print information about network memory usage.
nbr      : Print neighbor information.
ping     : Ping a network host.
pkt      : net_pkt information.
pmtu     : Show PMTU information.
ppp      : PPP information.
qbv      : Qbv commands
resume   : Resume a network interface
route    : Show network route.
sockets  : Show network sockets.
stats    : Show network statistics.
suspend  : Suspend a network interface
tcp      : Connect/send/close TCP connection.
udp      : Send/recv UDP packet
virtual  : Show/manipulate virtual network interfaces.
vlan     : Show VLAN information.
websocket : Print information about WebSocket connections.
wg       : Show information about the Wireguard VPN connections.
```



Onde está implementado?

Shell engine faz parte do Zephyr

Facilmente extensível pela aplicação ou bibliotecas próprias

Menus e funcionalidades habilitados por configurações de compilação

```
# prj.conf

# Shell
CONFIG_SHELL=y
CONFIG_SHELL_PROMPT_UART="iot-2026:~$ "

CONFIG_ADC_SHELL=y
CONFIG_GPIO_SHELL=y
CONFIG_INPUT_SHELL=y
CONFIG_LED_SHELL=y
CONFIG_I2C_SHELL=y
CONFIG_I3C_SHELL=y
CONFIG_SENSOR_SHELL=y
CONFIG_SENSOR_SHELL_STREAM=y
CONFIG_SPI_SHELL=y
CONFIG_NET_SHELL=y

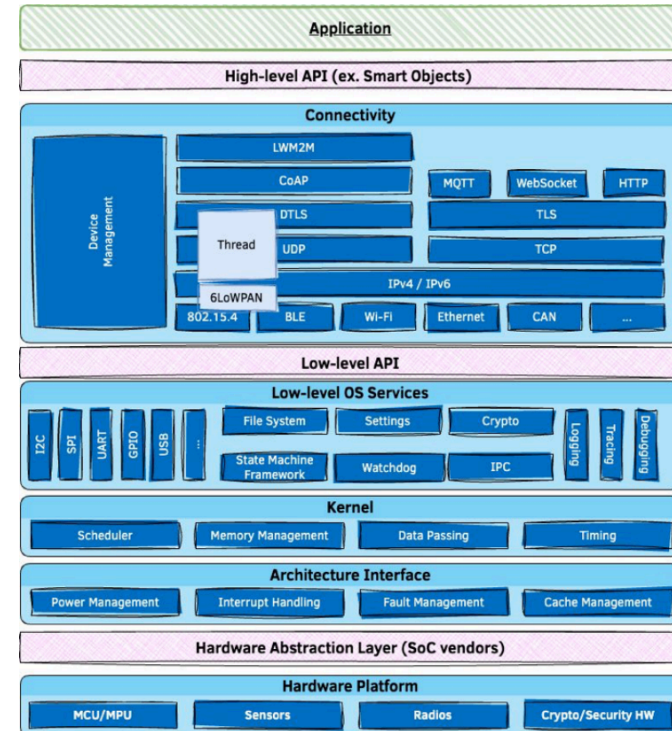
#
```



Onde está implementado?

Cada comando da Shell é um código de aplicação que utiliza:

- códigos built-in
- módulos importados
- repositórios próprios



© 2026 The Zephyr Project
Content made available under CC BY-SA 4.0



Bibliotecas e Configurações

20% programação

80% configuração



Aplicação desenvolvida com Zephyr RTOS

- repositório do Zephyr RTOS
 - ▷ kernel (thread, queue, mutex ...)
 - ▷ drivers (ADC, GPIO, I2C ...)
 - ▷ libs e subsys (shell, logging, protocolos, filesystem ...)
 - ▷ ferramental de compilação (west , .cmake , scripts ...)
 - ▷ boards (devicetree e configurações)
- módulos externos
 - ▷ suporte a targets e devices (vendors)
 - ▷ funcionalidades extras (bootloader, criptografia ...)
- repositórios próprios
 - ▷ bibliotecas
 - ▷ boards
 - ▷ aplicações





Uma *aplicação* precisa ser compilada para uma *board*





Código de *aplicação* pode ser
desacoplado do *hardware*



Gestão de hardware

board

Devicetree (.dts , .dtsi , .overlay)

- descrição do hardware **disponível**

- ▷ microcontrolador

- periféricos
- barramentos
- pinos

- ▷ componentes

compatible : driver com suporte ao device

aliases : possibilitam desacoplamento

```
// boards/nxp/frdm_mcxa156/frdm_mcxa156.dts
/ {
    compatible = "nxp,mcxa156", "nxp,mcx";

    aliases {
        led0 = &red_led;
        ambient-temp0 = &p3t1755;
    };

    leds {
        compatible = "gpio-leds";
        red_led: led_0 {
            gpios = <&gpio3 12 GPIO_ACTIVE_LOW>;
        };
    };

    &i3c0 {
        i3c-scl-hz = <DT_FREQ_M(4)>;

        p3t1755: p3t1755@4800000236152a0090 {
            compatible = "nxp,p3t1755";
        };
    };
};
```



Gestão de hardware

Aplicação

Devicetree (.overlay)

- configurações de **uso do hardware**
- habilitar/desabilitar devices e periféricos
- suporte a múltiplas boards
 - ▷ app.overlay
 - ▷ <board_1>.overlay
 - ▷ <board_2>.overlay

```
// app/boards/iot_2026_mcxa156.overlay
```

```
&i3c0 {  
    i3c-scl-hz = <DT_FREQ_M(1)>;  
};
```

```
&i2c0 {  
    status = "disabled";  
};
```

```
&i2c1 {  
    status = "okay";  
};
```

```
mpu6050: mpu6050@11 {  
    compatible = "invensense,mpu6050";  
    reg = <0x11>;  
    int-gpios = <&gpio3 0x01 0>;  
};  
};
```



Criar uma board

1. Encontrar uma board existente similar (Zephyr docs / supported boards)
2. Acessar pasta da board no repositório
3. Analisar sua DTS, configurações, customizações de compilação
4. Copiar definição da board para um módulo Zephyr em **repositório próprio**
5. Renomear e fatorar arquivos com o nome desejado para a board
6. Compilar aplicação para a nova board (*compatível com a board de referência*)
7. Customizar board para seu hardware

```
$ cd zephyr/boards/nxp/frdm_mcxa156
```

```
$ tree -L2
```

```
.
├── board.c
├── board.cmake
├── board.yml
├── CMakeLists.txt
├── doc
│   ├── frdm_mcxa156.webp
│   └── index.rst
├── dts
│   └── lpspi1_header.overlay
├── frdm_mcxa156_defconfig
├── frdm_mcxa156.dts
├── frdm_mcxa156-pinctrl.dtsi
├── frdm_mcxa156.yaml
├── Kconfig
└── Kconfig.frdm_mcxa156
```

```
cd zephyr/samples/basic/blink
```

```
west build -b frdm_mcxa156 -p
```

```
west build -b iot_2026_mcxa156 -p
```



Gestão de configurações

Kconfig - o mesmo utilizado pelo Linux

- board
 - ▷ defconfig
 - ▷ Kconfig.<board>
- aplicação
 - ▷ prj.conf
 - ▷ <board_1>.conf
 - ▷ <board_2>.conf
- menuconfig / guiconfig
 - ▷ pesquisa de opções
 - ▷ alteração rápida para testes
 - ▷ exportar configuração

```
# prj.conf
```

```
# Generic
CONFIG_LOG=y
CONFIG_REBOOT=y
```

```
# Peripherals
CONFIG_ADC=y
CONFIG_GPIO=y
```

```
# Shell
CONFIG_SHELL=y
CONFIG_SHELL_PROMPT_UART="iot-2026:~$ "
CONFIG_ADC_SHELL=y
```

```
# boards/iot_2026_mcx156.conf
```

```
# Enable I2C and GPIO expander
CONFIG_I2C=y
CONFIG_GPIO_PCA95XX=y
```

```
CONFIG_SHELL_PROMPT_UART="iot-2026_new:~$ "
```



menuconfig - Drivers

```
(Top) → Device Drivers
Zephyr Kernel Configuration
[*] Analog-to-Digital Converter (ADC) drivers --->
[ ] Audio drivers ----
[ ] Auxiliary (textual) display drivers ----
[ ] Battery-backed RAM (BBRAM) drivers ----
[ ] Biometrics drivers ----
[ ] External cache controller drivers ----
[ ] Controller Area Network (CAN) drivers ----
[ ] Battery charger drivers ----
-* Clock controller drivers --->
[ ] Comparator drivers ----
[*] Console drivers --->
[ ] Coredump pseudo-device drivers ----
[ ] Counter drivers ----
[ ] Cyclic Redundancy Check (CRC) drivers ----
[ ] Crypto drivers ----
[ ] Digital-to-Analog Converter (DAC) drivers ----
[ ] Digital Audio Interface (DAI) drivers ----
[ ] Debug drivers ----
[ ] Disk drivers ----
[ ] Display controller drivers ----
[ ] Direct Memory Access (DMA) drivers ----
[ ] Debug Port (DP) drivers [EXPERIMENTAL] ----
[ ] Error Detection and Correction (EDAC) drivers ----
[ ] Electrically Erasable Programmable Read-Only Memory (EEPROM) drivers ----
-* Entropy drivers --->
[ ] Enhanced Serial Peripheral Interface (eSPI) bus drivers ----
[ ] Ethernet drivers ----
↓↓↓↓↓↓↓↓↓↓↓↓
[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```



menuconfig - Shell

```
(Top) → Subsystems and OS Services
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑ Zephyr Kernel Configuration
[ ] Modbus support ----
[ ] Modem modules ----
    Networking ---->
[ ] Non Volatile Memory (NVMEM) ----
    Power Management ---->
    Platform Management Communications Infrastructure (PMCI) ---->
    Portability ----
[ ] Profiling tools ----
    Random Number Generators ---->
--* RTIO ---->
    SD ---->
[ ] Secure storage subsystem ----
[ ] Settings ----
--* Shell ---->
[ ] Statistics support
    Storage ---->
[ ] Task-level software watchdog ----
    Testing ---->
    Coverage ---->
[ ] Timing Functions
[ ] Tracing Support
[ ] USB Device Support ----
[ ] New USB device stack ----
[ ] USB host stack [EXPERIMENTAL] ----
[ ] USB-C Stack Support ----
[ ] ZBus support ----
[ ] Make tristate Kconfig options and an 'm' selection available

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                  [?] Symbol info      [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```



menuconfig - Shell

```
(Top) → Subsystems and OS Services → Shell → Shell backends
Zephyr Kernel Configuration
[*] <SHELL_BACKEND_SERIAL> Serial backend
(90) <SHELL_BACKEND_SERIAL_INIT_PRIORITY> Initialization priority
(iot-2026::~$ ) <SHELL_PROMPT_UART> Displayed prompt name
[*] <SHELL_BACKEND_SERIAL_INTERRUPT_DRIVEN> Interrupt driven
    <choice SHELL_BACKEND_SERIAL_API> Mode (Interrupt driven) --->
[ ] <SHELL_BACKEND_SERIAL_FORCE_TX_BLOCKING_MODE> Force blocking mode for TX
(8) <SHELL_BACKEND_SERIAL_TX_RING_BUFFER_SIZE> Set TX ring buffer size
(64) <SHELL_BACKEND_SERIAL_RX_RING_BUFFER_SIZE> Set RX ring buffer size
(100) <SHELL_BACKEND_SERIAL_LOG_MESSAGE_QUEUE_TIMEOUT> Log message drop timeout (in milliseconds)
(512) <SHELL_BACKEND_SERIAL_LOG_MESSAGE_QUEUE_SIZE> Log message queue size
    <choice> Initial log level limit (System limit (LOG_MAX_LEVEL)) --->
[ ] <SHELL_BACKEND_DUMMY> Dummy backend.

show-name mode enabled

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```



menuconfig - P3T1755

```
Jump to symbol/choice/menu/comment
p3t
DT_HAS_NXP_P3T1755_ENABLED(=y)
P3T1755(=y) "P3T1755 digital temperature sensor driver"
SHIELD_P3T1755DP_ARD_I3C(=n)
SHIELD_P3T1755DP_ARD_I3C(=n)
```

Type text to narrow the search. Regexes are supported (via Python's 're' module). The up/down cursor keys step in the list. [Enter] jumps to the selected symbol. [ESC] aborts the search. Type multiple space-separated strings/regexes to find entries that match all of them. Type Ctrl-F to view the help of the selected item without leaving the dialog.



menuconfig - P3T1755

```
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑      Symbol information
Help:

  Enable driver for the P3T1755 digital temperature sensor.

Direct dependencies (=y):
  DT_HAS_NXP_P3T1755_ENABLED(=y)
  && SENSOR(=y)

Default:
  - y

Kconfig definition, with parent deps. propagated to 'depends on'
=====

At drivers/sensor/nxp/p3t1755/Kconfig:6
Included via /home/matheus/coding/events/events-iot-2026-zephyr/external/zephyr/Kconfig:8 → Kconfig.zeph
Menu path: (Top) → Device Drivers → Sensor drivers

config P3T1755
    bool "P3T1755 digital temperature sensor driver"
    default y
    select I2C(=y)
    select I3C(=y)
    depends on DT_HAS_NXP_P3T1755_ENABLED(=y) && SENSOR(=y)
    help
        Enable driver for the P3T1755 digital temperature sensor.

[ESC/q] Return to menu      [/] Jump to symbol
```



menuconfig - LED

```
(Top) → Device Drivers → Light-Emitting Diode (LED) drivers
Zephyr Kernel Configuration
Max compiled-in log level for led (Default) --->
(3) <LED_LOG_LEVEL>
(90) LED initialization priority
[*] LED shell
- - X-Powers AXP192/AXP2101 LED driver (NEW)
- - DAC LED driver (NEW)
[*] GPIO LED driver
- - HT16K33 LED driver (NEW) --->
- - IS31FL319x LED driver (NEW)
- - IS31FL3216A LED driver (NEW)
- - IS31FL3733 LED driver (NEW)
- - LEDs group multi-color driver (NEW)
- - LP3943 LED driver (NEW)
- - LP50XX LED controller (NEW)
- - LP5562 LED driver (NEW)
- - LP5569 LED driver (NEW)
- - Arduino Modulino buttons LEDs (NEW)
- - NCP5623 LED driver (NEW)
- - nPM13xx LED driver (NEW)
- - PCA9533 LED driver (NEW)
- - PCA9633 LED driver (NEW)
- - PWM LED driver (NEW)
- - SCT2024 LED controller support (NEW)
- - TLC59108 LED driver (NEW)
- - Microchip XEC BB-LED driver (NEW)

show-all mode enabled

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[I] Quit (prompts for save) [D] Save minimal config (advanced)
```



Configurações da Aplicação Exemplo

```
# prj.conf

# Generic
CONFIG_LOG=y
CONFIG_REBOOT=y

# Peripherals
CONFIG_ADC=y
CONFIG_GPIO=y
CONFIG_INPUT=y
CONFIG_LED=y
CONFIG_I2C=y
CONFIG_I3C=y
CONFIG_SENSOR=y
CONFIG_SPI=y
```

```
# Shell
CONFIG_SHELL=y
CONFIG_SHELL_PROMPT_UART="iot-2026:~$ "
CONFIG_ADC_SHELL=y
CONFIG_GPIO_SHELL=y
CONFIG_INPUT_SHELL=y
CONFIG_LED_SHELL=y
CONFIG_I2C_SHELL=y
CONFIG_I3C_SHELL=y
CONFIG_SENSOR_SHELL=y
CONFIG_SENSOR_SHELL_STREAM=y
CONFIG_SPI_SHELL=y
CONFIG_NET_SHELL=y

# Networking
CONFIG_NETWORKING=y
CONFIG_NET_IPV4=y
CONFIG_NET_STATISTICS=y
CONFIG_NET_CONFIG_SETTINGS=y
CONFIG_NET_CONFIG_MY_IPV4_ADDR="192.0.2.1"
```



Zephyr Samples

Como implemento "x"?



Zephyr Samples

Aplicações funcionais de demonstração

- drivers
- kernel
- protocolos
- subsistemas
- atualização remota

Compatíveis com diversas boards

± 600 samples no Zephyr v4.4

```
$ cd zephyr/samples
$ tree -L1
.
├── application_development
├── basic
├── bluetooth
├── boards
├── cpp
├── data_structures
├── drivers
├── hello_world
├── index.rst
├── kernel
├── modules
├── net
├── philosophers
├── posix
├── regulator
├── sensor
├── subsys
├── synchronization
├── sysbuild
├── tfm_integration
└── userspace
```



Zephyr Samples - Drivers

```
$ tree -L1 -d drivers
```

```
drivers
```

```
|— adc  
|— audio  
|— biometrics  
|— can  
|— charger  
|— crc  
|— crypto  
|— dac  
|— display  
|— eeprom  
|— flash_shell  
|— gnss  
|— gpio  
|— haptics
```

```
drivers
```

```
|— i2c  
|— ipm  
|— lcd_hd44780  
|— lora  
|— modem  
|— pwm  
|— rtc  
|— smbus  
|— soc_flash_nand  
|— soc_flash_nrf  
|— spi_bitbang  
|— spi_flash  
|— stepper  
|— uart  
|— watchdog
```



Zephyr Samples - Networking

```
$ tree -L1 -d net
net
├── capture
├── cellular_modem
├── cloud
├── common
├── dhcpv4_client
├── dns_resolve
├── ethernet
├── ftp_client
├── ipv4_autoconf
├── latmon
├── lwm2m_client
├── mdns_responder
├── midi2
├── mqtt_publisher
├── mqtt_sn_publisher
└── ocpp
```

```
net
├── openthread
├── pkt_filter
├── prometheus
├── promiscuous_mode
├── ptp
├── qos
├── secure_mqtt_sensor_actuator
├── sockets
├── stats
├── syslog_net
├── telnet
├── tftp_client
├── virtual
├── wifi
├── wireguard
├── wpan_serial
└── zperf
```



Sample subsys/input/input_dump

```
// subsys/input/input_dump/src/main.c
/*
 * Copyright 2023 Google LLC
 *
 * SPDX-License-Identifier: Apache-2.0
 */

#include <stdio.h>

int main(void) {
    printf("Input sample started\n");
    return 0;
}
```

```
# prj.conf
CONFIG_LOG=y
CONFIG_LOG_MODE_MINIMAL=y

CONFIG_INPUT=y
CONFIG_INPUT_EVENT_DUMP=y
```

```
$ west build -b iot_2026_mcxa156 . -p
[163/163] Linking C executable zephyr/zephyr.elf
Memory region      Used Size  Region Size  %age Used
      FLASH:        22240 B         1 MB      2.12%
      RAM:           6704 B        120 KB      5.46%
      IDT_LIST:         0 B         32 KB      0.00%
```

```
$ west flash
$ picocom /dev/ttyACM0 -b 115200
*** Booting Zephyr OS build v4.4.0 ***
Input sample started
I: input event: dev=gpio_keys  code= 11 value=1
I: input event: dev=gpio_keys  code= 11 value=0
I: input event: dev=gpio_keys  code=  2 value=1
I: input event: dev=gpio_keys  code=  2 value=0
I: input event: dev=gpio_keys  code= 11 value=1
I: input event: dev=gpio_keys  code= 11 value=0
I: input event: dev=gpio_keys  code=  2 value=1
I: input event: dev=gpio_keys  code=  2 value=0
```



Zephyr west meta-tool



```
$ west -h
usage: west [-h] [-z ZEPHYR_BASE] [-v] [-q] [-V] <command> ...
```

The Zephyr RTOS meta-tool.

built-in commands for managing git repositories:

init:	create a west workspace
update:	update projects described in west manifest
status:	"git status" for one or more projects
grep:	run grep or a grep-like tool in one or more local projects

extension commands from project zephyr (path: external/zephyr):

boards:	display information about supported boards
build:	compile a Zephyr application
twister:	west twister wrapper
flash:	flash and run a binary on a board
debug:	flash and interactively debug a Zephyr application
attach:	interactively debug a board
spdx:	create SPDX bill of materials
robot:	run RobotFramework test suites
packages:	manage packages for Zephyr



Twister

Integrando ao DevOps



Descrição de testes para o twister

```
# samples/shell/sample.yaml
sample:
  name: Shell sample
tests:
  events-iot-2026.sample.shell:
    build_only: true
    tags:
      - LED
      - gpio
      - SPI
      - I2C
    integration_platforms:
      - native_sim/native
      - frdm_mcxa156
      - frdm_k82f
      - stm32f4_disco
      - nucleo_f302r8
      - iot_2026_mcxa156
```



Compilação e execução de testes

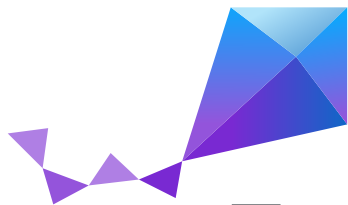
```
$ west twister -i --integration -T samples
INFO - Using Ninja..
INFO - Zephyr version: v4.4.0
INFO - Using 'zephyr/gnu' toolchain variant.
INFO - Writing JSON report /events-iot-2026-zephyr/twister-out/testplan.json

INFO - Adding tasks to the queue...
INFO - Added initial list of jobs to queue
INFO - Total complete:      6/6 100% built (not run): 6,
                                filtered:          0,
                                failed:             0,
                                error:              0

INFO - 0 of 6 executed test configurations passed (0.00%),
        6 built (not run), 0 failed, 0 errored, with no warnings in 17.93 seconds.
INFO - 0 of 0 executed test cases passed (0.00%) on 0 out of total 1474 platforms (0.00%).
INFO - 6 selected test cases not executed: 6 not run (built only).
INFO - 0 test configurations executed on platforms, 6 test configurations were only built.

INFO - Writing JSON report /events-iot-2026-zephyr/twister-out/twister.json
INFO - Run completed
```





Zephyr[®]

Devo começar a utilizar?



Características dos projetos

- O que o Zephyr disponibiliza faz sentido para meus projetos?
- Está entre os requisitos:
 - ▷ protocolos de rede
 - ▷ stack USB
 - ▷ interface gráfica
 - ▷ armazenamento de arquivo
- Os targets que pretendo utilizar são suportados?
- E os componentes periféricos, tem drivers?



Considerar

- Treinamentos e estudo para os desenvolvedores
- Tempo de aprendizado e adaptação
- Integração entre times para ampliar benefícios



Benefícios adicionais

- Possibilidade de testes da aplicação em outro hardware antes da chegada dos protótipos
- Bringup facilitado após a criação da board
- Diversas bibliotecas e drivers prontas



Como começar

1. Acessar documentação de Zephyr
2. Obter uma board com suporte nativo
3. Preparar o ambiente de desenvolvimento
4. Escolher um exemplo
5. Compilar
6. Gravar na placa
7. Utilizar
8. Interagir com o projeto:
 - habilitar mais funcionalidades
 - consultar samples
 - alterar código fonte
 - adicionar dispositivos na board e no software



Importante

O Zephyr tem complexidade alta porque atua em vários desafios complexos

- descrição de hardware
- kernel real time
- drivers
- build system
- desacoplamento do hardware
- simulação
- testes automatizados
- gestão de manifesto de projeto

Quem já precisou recriar esse ferramental (ou parte dele), vai reconhecer o valor que fornece e o tempo ganho



Dúvidas e comentários ?



 github.com/matheusmbar/events-iot-2026-zephyr

 [/matheusmbar](https://www.linkedin.com/company/matheusmbar)

 mail@matheusmbar.com

